AS5524: Computational Project Report

Colton Quirk

April 3, 2026

Table of contents

1 1D Problem **1**
1.1 Reconstruction . . . 1
1.1.1 Flat Reconstruction . . . 1
1.1.2 Linear Reconstruction . . . 2
1.2 Time-Stepping . . . 2
1.2.1 Euler Method . . . 2
1.2.2 2^{nd} Order Runge-Kutta . . . 3
1.2.3 4^{th} Order Runge-Kutta . . . 3
1.3 Comparison to Model . . . 4

2 2D Problems **4**
2.1 Blast Wave Example . . . 4
2.1.1 Plotting Method . . . 4
2.2 Custom Problems . . . 5
2.2.1 Blast Ring . . . 5
2.2.2 Kelvin-Helmholtz . . . 5

3 Animation **9**

4 Conclusion **9**

1 1D Problem

First I corrected the error with saving the simulation data from the `_write_numpy` function in `output.py`. This error came from the line

```
np.save(self._file_name, (V, g.x1))
```

This error was caused by the fact that `V` has shape $(3, 514)$ and `g.x1` has shape $(514,)$. `numpy` is not able to convert these shapes into a single array. The simple solution was to use the numpy function `np.savez(self._file_name, V=V, x1=g.x1)`. This function allows numpy to save the data despite having different shapes. I then modified the `plot_1d.py` function to read the `.npz` files.

As well just a little change there was an issue with the files being read in by `plot_1d.py` not sorted. This lead to the plots showing different simulation snapshots and in the wrong order. I replaced that with `files = sorted(glob.glob(f"{data_path}*.npz"))`. I used `glob` as it is an easy way to grab all files that follow a specific pattern.

1.1 Reconstruction

1.1.1 Flat Reconstruction

I first implemented the "flat" or Nearest Neighbors reconstruction algorithm.

```
# Return the neighboring values directly
L = V[:, :-1]
R = V[:, 1:]
```

1.1.2 Linear Reconstruction

Once I confirmed that flat reconstruction worked, I implemented the "linear" reconstruction algorithm.

I implemented the slope-limiter using `np.where()` this function makes it easy to implement the logic of the minmod algorithm.

$$\Delta\vec{V} = \text{minmod}(\vec{V}_i - \vec{V}_{i-1}, \vec{V}_{i+1} - \vec{V}_i)$$

```
# "Left" difference
delta_minus = V[:, 1:-1] - V[:, :-2]
# "Right" difference
delta_plus = V[:, 2:] - V[:, 1:-1]
```

$$\text{minmod}(a,b) = \begin{cases} a, & \text{if } |a| < |b| \text{ and } a \cdot b > 0 \\ b, & \text{if } |a| > |b| \text{ and } a \cdot b > 0 \\ 0, & \text{else} \end{cases}$$

```
delta_V = np.where(
        delta_minus * delta_plus > 0.0, # if a * b > 0.0
        np.where(
            np.abs(delta_minus) < np.abs(delta_plos),
            delta_minus, # return the value of a if |a| < |b|
            delta_plus   # return the value of b if |a| > |b|
        ),
        0.0 # otherwise, a * b < 0.0, return 0.0
    )
```

$$\vec{V}^n_{\pm} = \vec{V}^n \pm \frac{\Delta\vec{V}}{2}$$

$$\vec{V}^n_L \quad \vec{V}^n_R$$

It took me a while to understand how the boundary conditions were used within the program. Eventually I recognized that I was returning the "cell-centered" values rather than the values at the interface. Because of this I needed to slice one more value from each edge to align with the expected shape.

```
# Calculate the values at the interface of the cells
L = V[:, 1:-2] + 0.5 * delta_V[:, :-1]
R = V[:, 2:-1] - 0.5 * delta_V[:, 1:]
```

I then ran the 1D problem using the linear method which lead to some unexpected results which are discussed in Section 1.3

1.2 Time-Stepping

1.2.1 Euler Method

I first implemented the Euler time-stepping method.

$$y_{n+1} = y_n + f(y_n)\Delta t$$

The specific form used in this problem is:

$$\vec{U}^{n+1} = \vec{U}^n + \frac{\Delta t}{\Delta x}\vec{\mathcal{L}}^n$$

```
# Euler Method
U_new = U + dt * RHSOperator(U, grid, algorithms, timing) / dx
# Apply boundary conditions
grid.boundary(U_new, parameters)
```

1.2.2 2nd Order Runge-Kutta

Once I confirmed that the Euler time-stepping worked without error, I implemented the 2nd Order Runge-Kutta (RK2) time-stepping algorithm.

$$\begin{aligned} k_1 &= f(y_n) \\ k_2 &= f(y_n + k_1 \Delta t) \\ y_{n+1} &= y_n + \left(\frac{k_1 + k_2}{2}\right)\Delta t \end{aligned}$$

$$\begin{aligned} \vec{U}^* &= \vec{U}^n + \frac{\Delta t}{\Delta x}\vec{\mathcal{L}}^n \\ \vec{U}^{n+1} &= \frac{1}{2}\left(\vec{U}^n + \vec{U}^* + \frac{\Delta t}{\Delta x}\vec{\mathcal{L}}^*\right) \end{aligned}$$

```
# 2nd Order Runge-Kutta
# Calculate the intermediate state
U_temp = U + dt * RHSOperator(U, grid, algorithms, timing) / dx
# Apply boundary conditions
grid.boundary(U_temp, parameters)
# Based on the past and intermediate state calculate the new state
U_new = 0.5 * (U + U_temp + dt * RHSOperator(U_temp, grid, algorithms, timing) / dx)
# Apply boundary conditions
grid.boundary(U_new, parameters)
```

However once I implemented RK2 I ran into some difficulties. When comparing my simulation to the model solution, the simulation with RK2 deviated from the solution where Euler had not. This caused a lot of confusion as RK2 is a 2^{nd} order scheme and thus should be more accurate than Euler integration.

1.2.3 4th Order Runge-Kutta

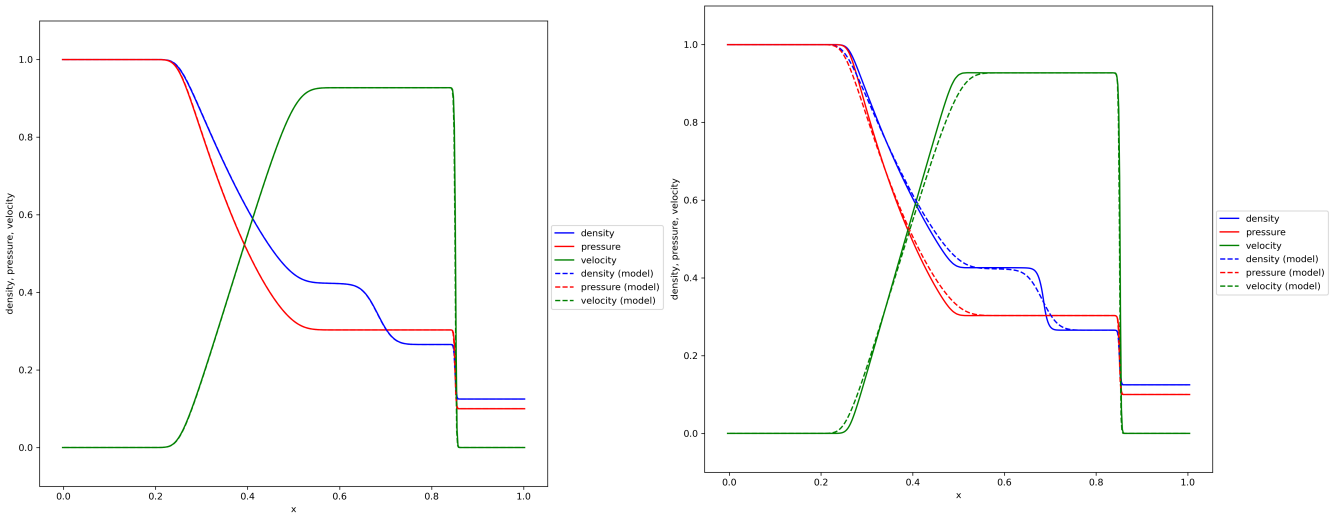
After implementing RK2 I wanted to implement the 4^{th} order Runge-Kutta (RK4) integration. I understood the method to calculate the intermediate timesteps from RK2. Thus I extended RK2 to calculate the 4 intermediate steps needed for RK4.

$$\begin{aligned} k_1 &= f(y_n, t_n) \\ k_2 &= f(y_n + k_1 \Delta t/2, t_n + \Delta t/2) \\ k_3 &= f(y_n + k_2 \Delta t/2, t_n + \Delta t/2) \\ k_4 &= f(y_n + k_3 \Delta t, t_n + \Delta t) \\ y_{n+1} &= y_n + \left(\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6}\right)\Delta t \end{aligned}$$

See the source code for my implementation of RK4.

1.3 Comparison to Model

The simulation that was best able to recreate the model solution was using the Euler Method and Flat Reconstruction. Figure 1a shows the final image of the simulation. Figure 1b shows the final frame of the 1D simulation using RK4 and linear reconstruction. The "more advanced" methods were not able to recreate the model solution as well as the simpler methods. This is likely due to the nature of the problem we are simulating. The linear reconstruction method makes the features a bit "sharper", with smaller transition regions between the various features.



(a) Euler time-stepping and flat reconstruction

(b) 4^{th} order Runge-Kutta time-stepping and linear reconstruction

Figure 1: Comparison of shock tube simulation results using various methods.

Given that I was able to recreate the model solution I moved on to the 2D simulations.

2 2D Problems

2.1 Blast Wave Example

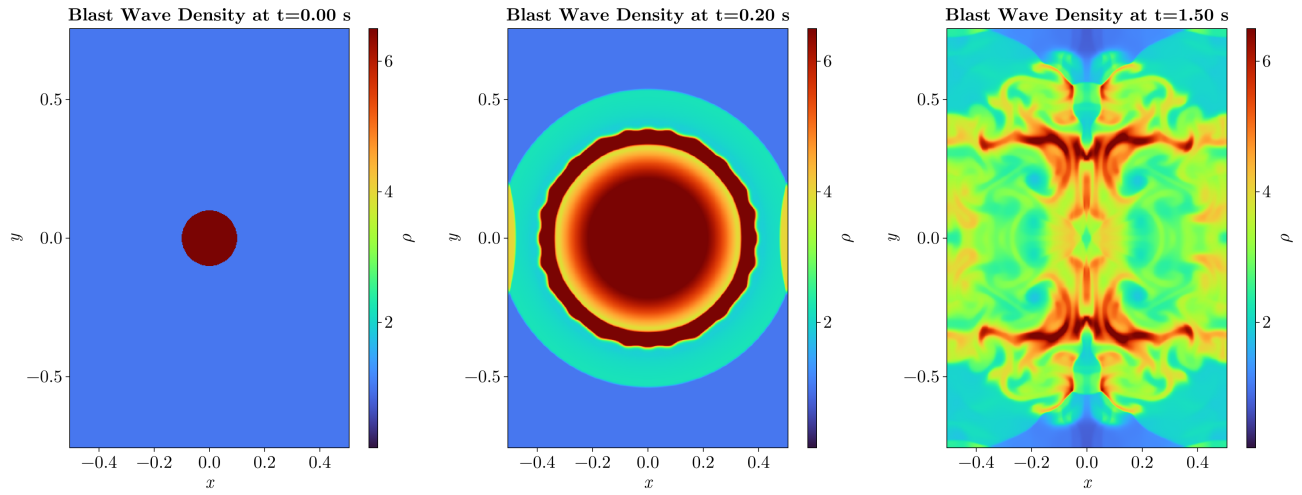
The first simulation was the Blast Wave problem described in the detailed project instructions. I changed the problem parameters to be:

- $-0.5 < x < 0.5$, $-0.75 < y < 0.75$
- Resolution: 400×600
- Boundary conditions are reciprocal everywhere.

These changes were motivated by the Athena Code Test "Spherical Blast Wave" example. I wanted to compare the blast wave example to the Athena example however there were issues with that which are discussed in Section 2.2.1. If the boundary conditions were "outflow" we would not get complex interactions between the waves in the fluid and the blast would only spread out. As well by changing the dimensions of x and y this leads to differences between the vertical and horizontal directions.

2.1.1 Plotting Method

I created `plot_2d.py` by copying `plot_1d.py` and modifying it. First by adding in multiple subplots that showed the different variables. This was a useful method to ensure that the initial conditions of the problem were set correctly. I used `pcolormesh` to display the images as I am more familiar with that plotting method than `imshow`. However, eventually I switched over to plotting in Julia when making the animations. Julia provides some very useful plotting features for videos as discussed in Section 3



(a) Initial conditions of the blast wave (b) Blast wave simulation at 0.2 seconds (c) Blast wave simulation at 1.5 seconds

Figure 2: The blast wave simulation at various times.

2.2 Custom Problems

Another change that I made was to save the data and plots based on the `Name` problem parameter. When switching between the custom problems there were issues with the new problems overwriting old data and plots. By using the name parameter in this way each problem is saved to a unique location. I changed `output.py` script to use the `Name` of the problem as another directory for the output folders.

I created two custom 2D simulations. The first was a recreation of the Athena Code Test "Spherical Blast Wave". The second was a recreation of the Athena Code Test "Kelvin-Helmholtz Instability".

2.2.1 Blast Ring

This problem is a variation on the initial blast wave problem. When comparing the blast wave Athena Code Test I noticed some discrepancies between the stated initial conditions and the provided demonstration. As Figure 3a shows the initial conditions for the Athena simulation were not an initial overdensity within $r < 0.1$.

Instead the simulation starts from a ring at $r = 0.1$ with the overdensity. I was able to implement this by creating a mask around $r = 0.1$ and setting the overdensity there. After some experimenting I found that a ring with width $\Delta r = 0.005$ reasonably recreated the Athena simulation. Figure 3b shows this initial setup.

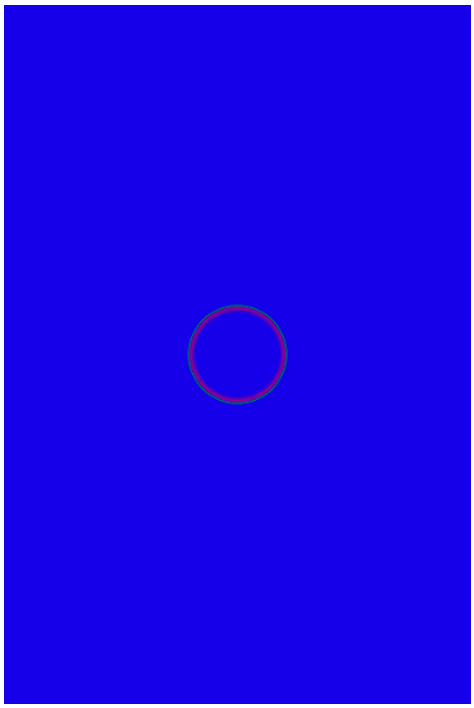
The main comparison for this problem is the creation of "fingers" within the fluid that do not dissipate after creation. As well as the density being symmetric horizontally and vertically throughout the simulation.

2.2.2 Kelvin-Helmholtz

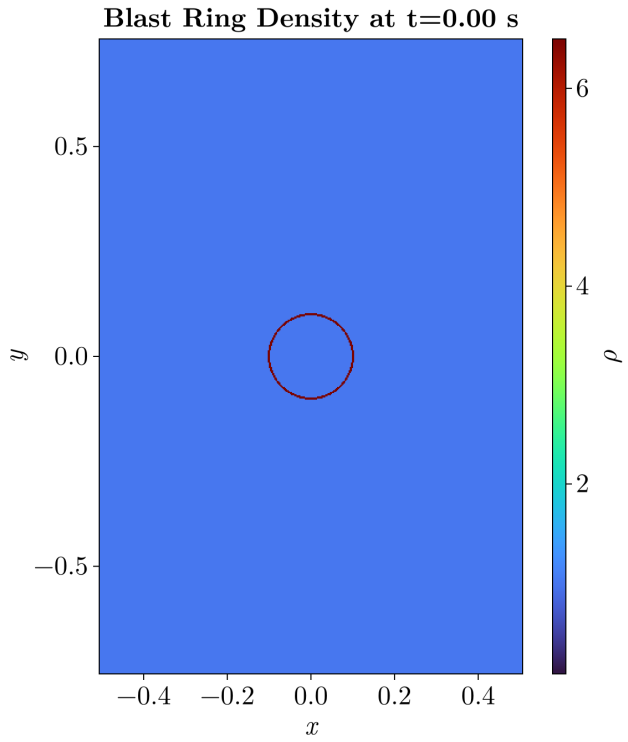
For the second custom problem I decided to look at the Kelvin-Helmholtz Instability. This was mentioned in the lectures and produced beautiful videos. The initial conditions I used were based on the Athena Code Test.

- $-0.5 < x < 0.5$, $-0.75 < y < 0.75$
- Resolution: 512×512
- Pressure: 2.5 everywhere
- $|y| > 0.25$ has density 1.0 and $v_x = -0.5$
- $|y| < 0.25$ has density 2.0 and $v_x = 0.5$
- $v_y = 0.0$ everywhere
- Reciprocal boundary conditions everywhere

However with just these initial conditions the instabilities did not form. I had missed an important line in the initial conditions:

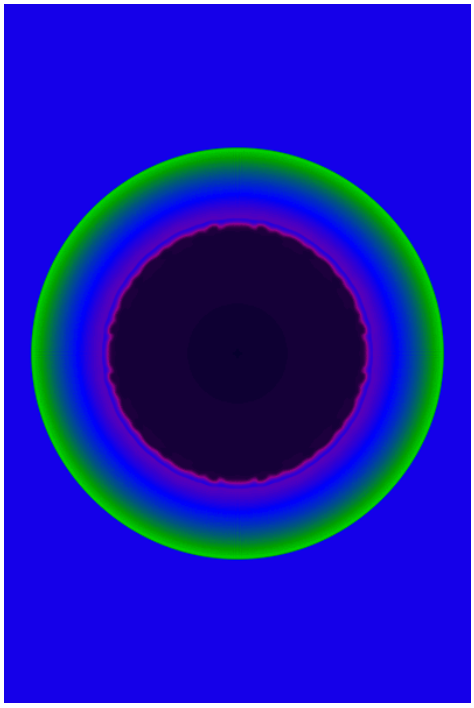


(a) Athena Code Test “Blast Wave” initial conditions

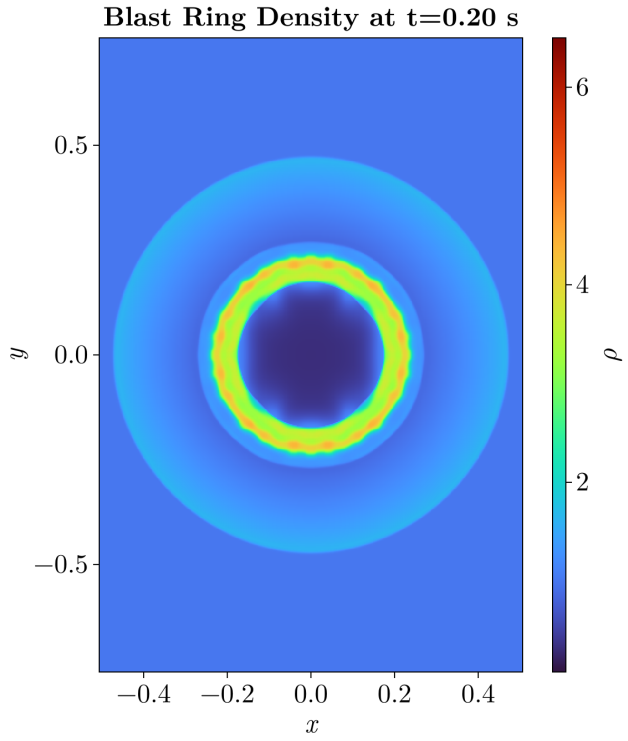


(b) My blast ring initial condition

Figure 3: The initial conditions of the blast ring simulation.



(a) Athena blast wave simulation



(b) Blast ring simulation

Figure 4: Comparison of Athena simulation and my simulation at $t = 0.2$s.

Kelvin-Helmholtz Initial Condition

To seed the instability, we add random numbers to both the x- and y-components of the velocity with peak-to-peak amplitude of 0.01.

Without these random velocities the instabilities did not form and the flow continued smoothly. Figure 5 shows the initial conditions of the Kelvin-Helmholtz Instability. While it is not visible in the v_x subplot the v_y subplot shows the initial random velocities.

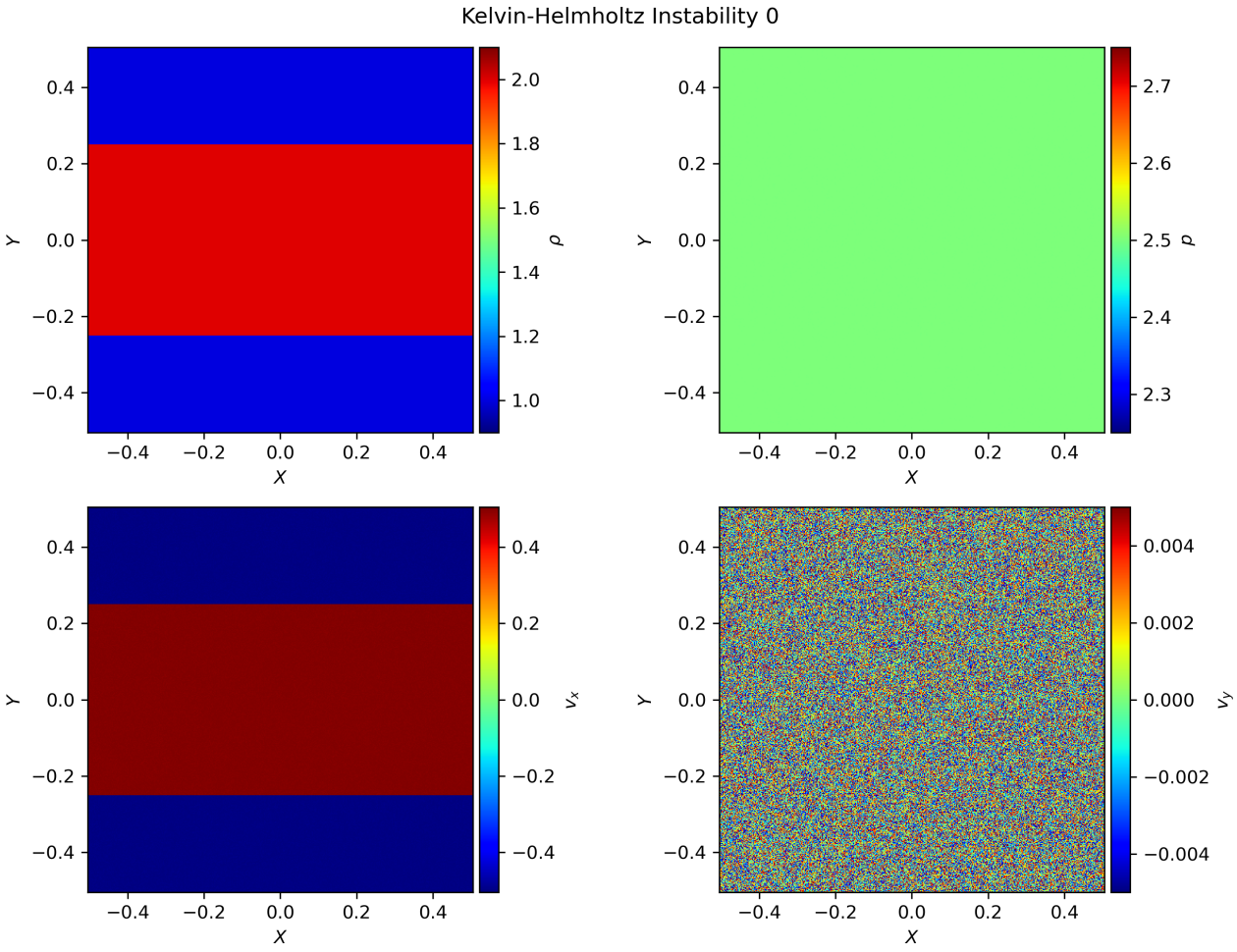
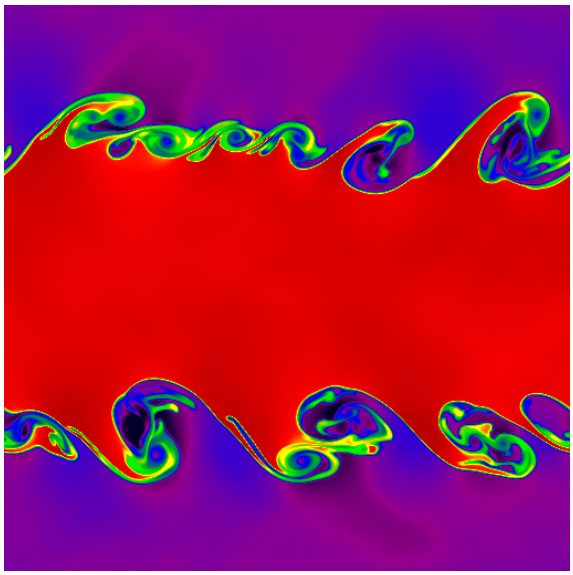


Figure 5: Initial conditions of the Kelvin-Helmholtz Instability simulation.

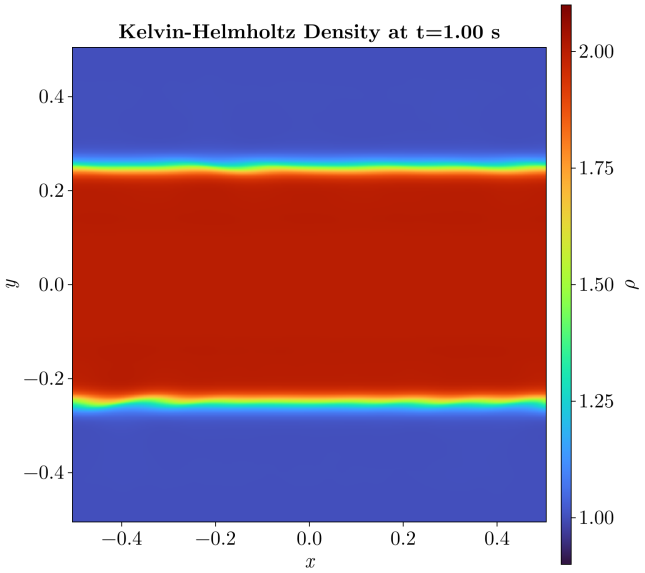
Initially I ran this simulation for a simulation time of $t = 1.0$s. However the instabilities were just beginning to form, unlike in the Athena code test where the instabilities were already visible at one second, see Figure 6.

But as it seemed the instabilities were forming I decided to increase the max time parameter to five seconds to match the Athena simulation. Increasing the simulation time made the simulation take approximately 2.5 hours, using the linear reconstruction method and RK4. Figure 7 shows the instabilities forming at two seconds.

The simulation does achieve the expected instabilities. However when comparing to the Athena code test there is a lack of detail. There are a few potential reasons for these differences. The main one comes from Athena using more advanced solvers that preserve those small details. The linear reconstruction method is not able to retain those small vortices. As well Athena used the "hllc" Riemann solver while this simulation used "hll". From what I have seen the hllc solver utilizes a more accurate model of the waves which retains that extra detail.



(a) Athena Kelvin-Helmholtz Instability simulation



(b) My Kelvin-Helmholtz Instability

Figure 6: Comparison of Kelvin-Helmholtz Simulations at $t = 1.0$s of simulation time.

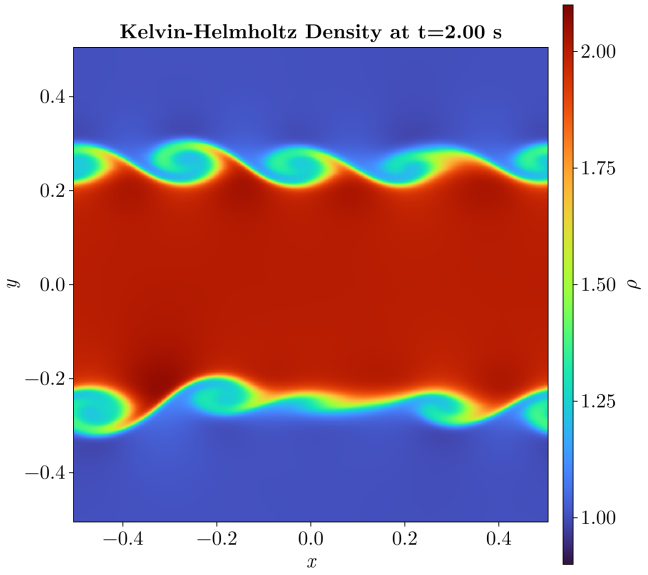


Figure 7: Kelvin-Helmholtz Instability simulation at two seconds.

3 Animation

I attempted many methods for animating the results:

- FFmpeg to convert the generated plots to a .mp4 file.
- yt-project to make the plots and then stitch them together with FFmpeg.
- `matplotlib.animation` library to automatically make the video
- julia plotting as making animations is very easy
  - animating with the `@animate` macro using `Plots`
  - animating using `Observables` and `CairoMakie`

The method that I eventually settled on was to make the animations in Julia using the CairoMakie package. This package makes use of `Observables` that are variables that julia monitors and uses the GPU to change in place. This method make videos faster than the standard method from matplotlib or making the frames individually and stitching them together.

At first I just plotted the densities using a heatmap to ensure that the videos were generating properly. Once that was confirmed to work I included the other variables pressure, p, and the velocity fields.

The velocity animation took the longest to make work. Initially I wanted to use streamlines to show the velocity flow. However when attempting this in both python and julia there were a few issues. The main one being that the streamlines between frames were not consistent. So the video did not provide an understanding of how the flow changes over time. I changed from using streamlines to using `arrows2d` which is equivalent to matplotlibs `quiver` function. This shows the velocity field over time and by sampling specific points within the velocity field it was consistent.

The pressure was made using the same heatmap method as for the density but with a different colorscale. At first I set the pressure colorscale based on the density colorscale and dividing by the value of γ used in that simulation. This generally worked but did not provide all the detail I wanted, so eventually I just tried different values to reveal more detail.

For the animation of the Kelvin-Helmholtz problem I wanted to see the formation of larger vortices so I increased the simulation time to 10 seconds. This took approximately six hours to run using linear reconstruction and RK4. The vortices did grow over time as Figure 8 shows the last frame of the simulation.

4 Conclusion

This was a interesting project allowing us to dive deeper into computational fluid dynamics (CFD). CFD is a topic I have wanted to explore for a while but have not had the chance to before. It was useful to focus on just a few aspects of the code but being able to see how the various parts of the simulation fit together. I have certainly gained a deeper understanding and appreciation of what it takes to develop a computational fluid dynamics system. As well it was fun making my own simulations and producing beautiful plots and movies.

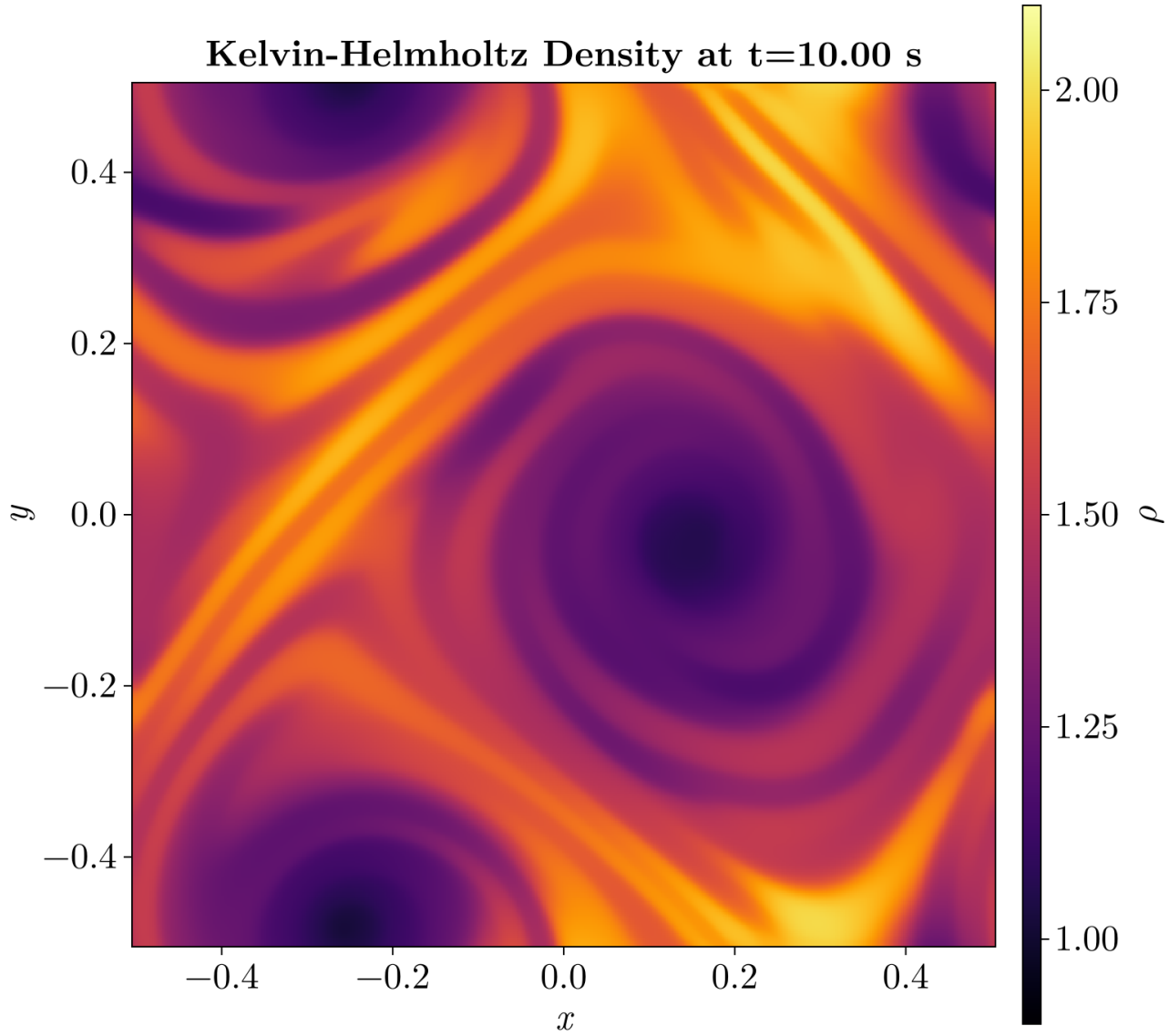


Figure 8: Kelvin-Helmholtz Instability simulation at 10 seconds.